

INTEGRACIÓN DE PROYECTOS EN REPOSITARIOS DE GITHUB

GUÍA METODOLÓGICA PARA EL DESPLIEGUE DE PROYECTOS
STM32CUBEIDE Y CONTROL DE VERSIONES MODULAR

Autor:
Rodrigo CASTILLEJOS

Proyecto:
Control de Dispositivos Embebidos

May 31, 2026

RESUMEN

Este documento describe de manera exhaustiva el procedimiento técnico para integrar de manera limpia un proyecto de software externo (desarrollado de forma local fuera del espacio del control de versiones) dentro de un repositorio de GitHub previamente establecido. Para ilustrar el flujo de trabajo práctico, se detalla paso a paso la incorporación del proyecto firmware NUCLEO-F303_TMP117 (compilado en STM32CubeIDE en la unidad D:) dentro del repositorio modular de librerías locales ubicado en la ruta `C:\Users\casti\OneDrive\Documentos\GitHub\Libraries\Libraries`.

A lo largo del documento se abordan consideraciones clave como el particionamiento de los espacios de trabajo, la transferencia recursiva y limpia de archivos mediante terminal de línea de comandos en Windows PowerShell, las directrices óptimas de filtrado mediante el uso y fusión de archivos `.gitignore` para evitar el almacenamiento innecesario de archivos binarios e intermedios de compilación pesados, y la secuencia completa de comandos Git en la rama de desarrollo `feature/update-TMP117` para la fase de preparación, registro y publicación remota.

TABLA DE CONTENIDOS

1	Introducción y Objetivos	4
2	Arquitectura de Directorios y Espacios de Trabajo	5
3	Copia Física de Archivos mediante PowerShell	6
3.1	Comandos de Transferencia	6
3.2	Método Gráfico mediante el Explorador de Archivos (Sin Consola)	6
4	Configuración y Fusión de Reglas .gitignore	7
4.1	El archivo .gitignore Existente	7
4.2	Fusión con Reglas de Compilación Embebidas	7
5	Operaciones con Git (Add, Commit y Push)	9
5.1	Flujo de Comandos de Git	9
5.2	Explicación de Pasos Críticos	9
6	Conclusiones y Buenas Prácticas	10

1 INTRODUCCIÓN Y OBJETIVOS

En el desarrollo profesional de firmware para sistemas embebidos, es habitual mantener dos espacios de trabajo separados: uno local dedicado a la experimentación activa, prueba de hardware y compilación rápida (usualmente ubicado en una ruta temporal de desarrollo del IDE), y un segundo espacio de trabajo integrado en la estructura organizada de un sistema de control de versiones centralizado como Git.

La correcta integración de proyectos generados por entornos de desarrollo como STM32CubeIDE dentro de un repositorio de Git requiere cumplir con objetivos metodológicos fundamentales:

- **Seguridad y Trazabilidad:** Asegurar que la lógica del firmware, configuraciones del microcontrolador y dependencias queden almacenadas en la rama correspondiente del repositorio (por ejemplo, `feature/update-TMP117`).
- **Limpieza de Repositorios (Exclusión de Compilación):** Evitar estrictamente la carga en la nube de archivos binarios temporales pesados (como `.elf`, `.hex`, y archivos objeto `.o`) generados en cada compilación por la cadena de herramientas GCC del IDE.
- **Compartición de Configuración:** Compartir los parámetros lógicos esenciales del entorno de compilación (como la carpeta `.settings` o los archivos de proyecto del IDE `.project` y `.cproject`) para garantizar que cualquier otro desarrollador pueda clonar e importar el proyecto de forma inmediata y sin reconfiguraciones.

Esta guía metodológica detalla precisamente cómo lograr una transferencia de proyecto robusta, ordenada e indolora cumpliendo con estas normas críticas de diseño de software.

2 ARQUITECTURA DE DIRECTORIOS Y ESPACIOS DE TRABAJO

Para llevar a cabo el proceso de manera segura, se definen dos rutas clave en el almacenamiento del sistema operativo Windows:

DIRECTORIO DE TRABAJO LOCAL (ORIGEN): D:\STM32_Projects\NUCLE0-F303-TMP117

Es la carpeta donde reside el proyecto físico que se está desarrollando y probando con el microcontrolador en STM32CubeIDE. Es una ruta fuera del control de Git.

DIRECTORIO DEL REPOSITORIO GIT (DESTINO): C:\Users\casti\OneDrive\Documentos\GitHub\Libraries\Libraries

Es la raíz física del repositorio local donde se encuentra la carpeta oculta .git y la estructura modularizada de librerías en lenguaje C.

La estructura de carpetas final que se desea obtener dentro del repositorio se describe en la figura 1.

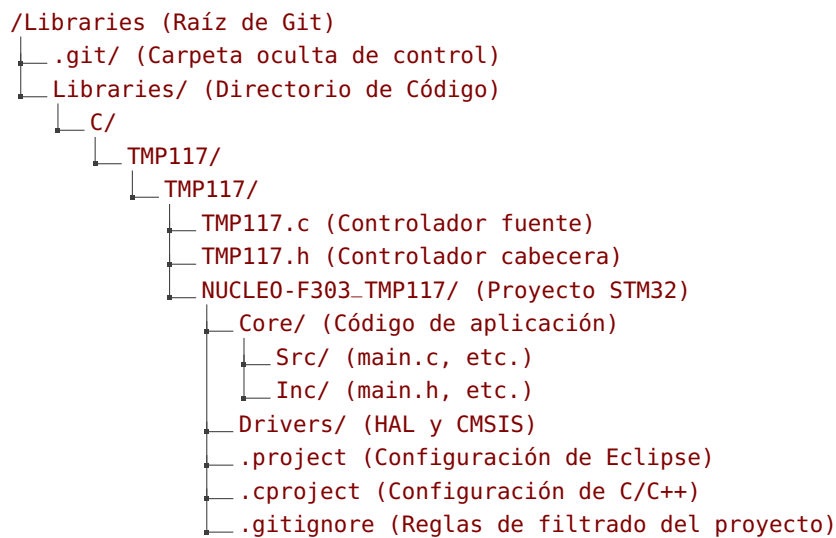


Figura 1: Estructura final del proyecto modularizado dentro del repositorio.

3 COPIA FÍSICA DE ARCHIVOS MEDIANTE POWERSHELL

El primer paso consiste en transferir toda la estructura de archivos desde el directorio de compilación local en el disco D: hacia la subcarpeta del sensor de temperatura TMP117 del repositorio en el disco C:.

Para realizar esto con precisión evitando la omisión accidental de archivos ocultos o del sistema, se recomienda utilizar el entorno de línea de comandos **Windows PowerShell**.

3.1 Comandos de Transferencia

Abre PowerShell y ejecuta la siguiente secuencia de comandos:

```
# 1. Crear el directorio de destino exacto en el repositorio de Git
New-Item -ItemType Directory -Force -Path "C:\Users\casti\OneDrive\Documentos\GitHub\Libraries\Libraries\C\TMP117\
TMP117\NUCLE0-F303_TMP117"

# 2. Copiar de forma recursiva y forzada todo el contenido del proyecto
Copy-Item -Path "D:\STM32_Projects\NUCLE0-F303_TMP117\*" -Destination "C:\Users\casti\OneDrive\Documentos\GitHub\
Libraries\Libraries\C\TMP117\TMP117\NUCLE0-F303_TMP117" -Recurse -Force
```

Explicación de modificadores en PowerShell

- **-Force:** Obliga a la creación de carpetas o sobrescritura de archivos existentes sin pedir confirmaciones interactivas.
- **-Recurse:** Indica a Copy-Item que debe procesar recursivamente todas las subcarpetas y archivos en el árbol del proyecto origen de forma automática.
- El asterisco (*) al final de la ruta origen copia los elementos contenidos en ella y no la carpeta padre por duplicado.

3.2 Método Gráfico mediante el Explorador de Archivos (Sin Consola)

Si prefieres no utilizar la consola de comandos de PowerShell, puedes realizar la transferencia de forma gráfica utilizando el **Explorador de Archivos de Windows**. Este método es más intuitivo, pero requiere especial atención a los archivos ocultos de configuración del IDE.

Sigue estos pasos para realizar la copia:

1. **Copiar el proyecto origen:** Abre el Explorador de Archivos, navega hasta la carpeta D:\STM32_Projects\, haz clic derecho sobre la carpeta NUCLE0-F303_TMP117 y selecciona **Copiar** (o presiona **Ctrl**+**C**).
2. **Pegar en el destino:** Navega hasta la ruta del repositorio local en: C:\Users\casti\OneDrive\Documentos\GitHub\Libraries\Libraries\C\TMP117\TMP117\ Haz clic derecho en un espacio vacío dentro de esa carpeta y selecciona **Pegar** (o presiona **Ctrl**+**V**).

Advertencia sobre Archivos Ocultos en el Explorador

Los entornos de desarrollo basados en Eclipse (como STM32CubeIDE) almacenan metadatos y configuraciones cruciales en archivos y carpetas ocultas cuyos nombres inician con un punto (por ejemplo, .project, .cproject y la carpeta .settings/).

Si utilizas el Explorador de Archivos, asegúrate de tener activada la opción **"Elementos ocultos"** (en la pestaña *Vista* del explorador) para garantizar que estos elementos se copien de manera correcta. Sin ellos, el proyecto no podrá ser importado de vuelta en el IDE por otros usuarios.

4 CONFIGURACIÓN Y FUSIÓN DE REGLAS .GITIGNORE

Un aspecto fundamental del control de versiones en firmware es evitar que archivos generados dinámicamente entren al repositorio. Los proyectos de STM32CubeIDE contienen archivos autogenerados que cambian en cada compilación. Subirlos causa *commits* redundantes masivos y desperdicia espacio.

4.1 El archivo .gitignore Existente

Si el proyecto ya contenía un archivo .gitignore básico al momento de la copia con las siguientes directrices:

```
1 build
2 mx.scratch
3 !.settings
```

- build: Ignora la carpeta temporal utilizada en sistemas de construcción con CMake.
- mx.scratch: Archivo temporal generado por STM32CubeMX para rastrear cambios en la generación del hardware.
- !.settings: El signo de exclamación (!) es una regla de exclusión negativa; indica que ****sí**** se debe realizar el seguimiento de la carpeta de configuraciones de eclipse (.settings), lo cual es correcto para guardar las vistas y perfiles del programador.

4.2 Fusión con Reglas de Compilación Embebidas

Para garantizar la limpieza completa ante una compilación típica basada en la herramienta interna de STM32CubeIDE (que suele generar la carpeta Debug o Release), debemos fusionar la lista de exclusiones añadiendo reglas para bloquear ejecutables y archivos de objeto:

El contenido final del archivo C:\Users\casti\OneDrive\Documentos\GitHub\Libraries\Libraries\C\TMP117\TMP117\NUCLEO-F303-TMP117\.gitignore debe configurarse de la siguiente manera:

```
1 build
2 mx.scratch
3 !.settings
4
5 # Excluir carpetas de compilación típicas de STM32CubeIDE
6 Debug/
7 Release/
8
9 # Excluir ejecutables y archivos intermedios de GCC
10 *.elf
11 *.bin
12 *.hex
13 *.map
14 *.list
15 *.o
16 *.su
17 *.d
18 *.o.d
```

Por qué excluir *.elf y *.o

Los archivos `.elf` y `.hex` son los binarios finales compilados listos para cargarse al microcontrolador. Pueden llegar a medir varios megabytes y cambian cada vez que modificas aunque sea una línea de comentarios en el código. Excluirlos previene el crecimiento exponencial e innecesario de la base de datos de Git.

5 OPERACIONES CON GIT (ADD, COMMIT Y PUSH)

Una vez que los archivos han sido físicamente copiados y el archivo `.gitignore` se encuentra debidamente fusionado y guardado, se debe proceder a registrar y subir los cambios a la rama remota de GitHub.

5.1 Flujo de Comandos de Git

Abre una terminal de Git Bash (o usa la terminal en VS Code) en la raíz física del repositorio y ejecuta los siguientes comandos de forma secuencial:

```
# 1. Navegar a la raíz física del repositorio local (donde se encuentra .git)
cd C:\Users\casti\OneDrive\Documentos\GitHub\Libraries\Libraries

# 2. Verificar el estado del repositorio y listar archivos no rastreados
git status

# 3. Cambiarse a la rama de desarrollo específica del sensor
git checkout feature/update-TMP117

# 4. Añadir el proyecto recién copiado de forma relativa al área de preparación
git add C/TMP117/TMP117/NUCLEO-F303-TMP117

# 5. Confirmar los cambios localmente con un mensaje descriptivo
git commit -m "Add NUCLEO-F303-TMP117 STM32CubeIDE project"

# 6. Subir la rama y los cambios al repositorio remoto en GitHub
git push origin feature/update-TMP117
```

5.2 Explicación de Pasos Críticos

1. **Ubicación de la Raíz:** Es fundamental ejecutar los comandos desde `Libraries\Libraries`. Si se ejecutan en una ruta superior o incorrecta, Git arrojará la advertencia fatal: `not a git repository`.
2. **Uso de Rutas Relativas:** Dado que ya te encuentras en la raíz del repositorio, la ruta en el comando `git add` inicia directamente con `C/TMP117/...` y no incluye el prefijo de la carpeta del repositorio.
3. **Verificación con `git status`:** Al ejecutar `git status` antes de hacer el commit, notarás que Git solo registrará el archivo `.gitignore`, los archivos del proyecto (`.project`, `.cproject`) y la carpeta de código `Core/`. Las carpetas `Debug/` o `build/` no deben aparecer en la lista gracias a las reglas configuradas en el paso anterior.

El comando `git push origin`

El comando `git push origin feature/update-TMP117` indica a Git que debe enviar el historial de commits locales de tu rama activa hacia el servidor remoto configurado con el alias `origin` (que apunta a tu cuenta de GitHub) directamente en la rama remota llamada `feature/update-TMP117`.

6 CONCLUSIONES Y BUENAS PRÁCTICAS

Integrar proyectos externos a GitHub es una tarea recurrente en el desarrollo de software que requiere orden y apego a estándares de Git para evitar "ensuciar" el repositorio. Siguiendo este flujo de trabajo, se logran las siguientes buenas prácticas en la gestión de firmware:

- **Independencia de Entornos de Compilación:** Al mantener el proyecto configurado únicamente con sus archivos de orígenes y directivas de exclusión, cualquier miembro del equipo puede importar el proyecto en su respectivo STM32CubeIDE de manera local.
- **Historial de Commits Limpio:** Al excluir archivos compilados en el archivo `.gitignore`, cada commit en Git representará exclusivamente cambios en el código lógico fuente (`.c` y `.h`), haciendo que las revisiones de código sean claras, concisas y fáciles de analizar.
- **Control de Ramas Organizado:** Trabajar bajo ramas de características (como `feature/update-TMP117`) en lugar de directamente sobre la rama principal (`main` o `master`) permite experimentar, desarrollar y probar de forma segura sin peligro de romper código estable hasta que los cambios estén listos para una fusión (*Pull Request*) definitiva.